Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Модели решения задач в ИС»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:

Студент 3 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Допускается применение математических и графических библиотек (NumPy, Matplotlib).

Использование ML-фреймворков (PyTorch, Keras и т.д.) запрещено.

Вариант:

№	n	Task
17	9	OR

Ход работы

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

N = 9

X_all = np.array(list(itertools.product([0, 1], repeat=N)), dtype=float)
y_all = np.all(X_all == 0, axis=1).astype(float)
```

```
print(f"Всего наборов: {len(X_all)}")
print(f"Класс 1: {int(y_all.sum())}  |  Класс 0: {int((1-y_all).sum())}")

np.random.seed(42)
idx_pos = np.where(y_all == 1)[0]
idx_neg = np.where(y_all == 0)[0]

neg_perm = np.random.permutation(idx_neg)
split_neg = int(0.7 * len(idx_neg))
train_neg, test_neg = neg_perm[:split_neg], neg_perm[split_neg:]

train_idx = np.concatenate([idx_pos, train_neg])
test_idx = test_neg
np.random.shuffle(train_idx)

X_train, y_train = X_all[train_idx], y_all[train_idx]
X_test, y_test = X_all[test_idx], y_all[test_idx]

print(f"Обучающая: {len(X_train)} (класс 1: {int(y_train.sum())})")
print(f"Тестовая:  {len(X_test)} (класс 1: {int(y_test.sum())})")

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def total_error(X, y, w, b, w_pos=1.0):
    yp = sigmoid(X @ w + b)
    weights = np.where(y == 1, w_pos, 1.0)
    return 0.5 * np.sum(weights * (y - yp) ** 2)

def accuracy(X, y, w, b):
    return np.mean((sigmoid(X @ w + b) >= 0.5).astype(float) == y)

def train_perceptron(X_tr, y_tr, X_te, y_te,
                     alpha_mode='fixed', alpha=0.3,
                     max_epochs=6000, tol=1e-4,
                     w_pos= 50.0):

    rng = np.random.default_rng(0)
    w = rng.uniform(0.5, 1.0, size=N)
    b = -float(N) + 0.5

    train_errors, test_errors = [], []
    lr = alpha
```

```
    prev_err = float('inf')

    for epoch in range(max_epochs):
        for i in rng.permutation(len(X_tr)):
            xi, yi = X_tr[i], y_tr[i]
            out = sigmoid(xi @ w + b)

            weight_i = w_pos if yi == 1 else 1.0
            delta = weight_i * (yi - out) * out * (1 - out)

            w += lr * delta * xi
            b += lr * delta

        err_tr = total_error(X_tr, y_tr, w, b, w_pos)
        err_te = total_error(X_te, y_te, w, b, w_pos)

        train_errors.append(err_tr)
        test_errors.append(err_te)

        if alpha_mode == 'adaptive':
            if err_tr < prev_err:
                lr = min(lr * 1.05, 2.0)
            else:
                lr = max(lr * 0.5, 1e-5)

        prev_err = err_tr

        if err_tr < tol:
            print(f"Сошлось за {epoch+1} эпох")
            break

    return w, b, train_errors, test_errors

print("\n=== Эксперимент А: фиксированный шаг ===")
wA, bA, tr_errA, te_errA = train_perceptron(
    X_train, y_train, X_test, y_test,
    alpha_mode='fixed', alpha=0.3)

print(f"Точность (обучающая): {accuracy(X_train, y_train, wA, bA)*100:.2f}%")
print(f"Точность (тестовая):  {accuracy(X_test, y_test, wA, bA)*100:.2f}%")
print(f"Точность (полная):    {accuracy(X_all, y_all, wA, bA)*100:.2f}%")

print("\n=== Эксперимент Б: адаптивный шаг ===")
```

```
wB, bB, tr_errB, te_errB = train_perceptron(
    X_train, y_train, X_test, y_test,
    alpha_mode='adaptive', alpha=0.2)

print(f"Точность (обучающая): {accuracy(X_train, y_train, wB, bB)*100:.2f}%")
print(f"Точность (тестовая):  {accuracy(X_test, y_test, wB, bB)*100:.2f}%")
print(f"Точность (полная):    {accuracy(X_all, y_all, wB, bB)*100:.2f}%")

print("\n=== Веса (фиксированный шаг) ===")
for i, wi in enumerate(wA):
    print(f"w{i+1} = {wi:.4f}")
print(f"b = {bA:.4f}")

print("\n=== Веса (адаптивный шаг) ===")
for i, wi in enumerate(wB):
    print(f"w{i+1} = {wi:.4f}")
print(f"b = {bB:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(tr_errA, label="Train fixed")
plt.plot(te_errA, label="Test fixed")
plt.plot(tr_errB, label="Train adaptive")
plt.plot(te_errB, label="Test adaptive")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.legend()
plt.grid()
plt.savefig("convergence.png", dpi=150)
plt.close()

print("\nГрафик сохранён: convergence.png")

def predict(x_vec, w, b):
    x = np.array(x_vec, dtype=float)
    prob = float(sigmoid(x @ w + b))
    return prob, int(prob >= 0.5)

print("\n=== РЕЖИМ РАБОТЫ ===")
print(f"Введите {N} чисел (0 или 1), либо q для выхода")

while True:
    raw = input("Вход: ").strip()
    if raw.lower() == 'q':
```

```
        break

    parts = raw.split()
    if len(parts) != N:
        print("Ошибка количества значений")
        continue

    try:
        vec = [int(p) for p in parts]
    except:
        print("Ошибка ввода")
        continue

    if any(v not in (0, 1) for v in vec):
        print("Только 0 и 1")
        continue

    prob, cls = predict(vec, wA, bA)
    true_val = int(all(v == 0 for v in vec))

    print(f"Вероятность: {prob:.4f}")
    print(f"Класс: {cls} | Истина: {true_val}\n")
```

Вывод программы:

```
Всего наборов: 512
Класс 1: 1  |  Класс 0: 511
Обучающая: 358 (класс 1: 1)
Тестовая:  154 (класс 1: 0)

=== Эксперимент А: фиксированный шаг ===
Точность (обучающая): 100.00%
Точность (тестовая):  99.35%
Точность (полная):    99.80%

=== Эксперимент Б: адаптивный шаг ===
Точность (обучающая): 100.00%
Точность (тестовая):  99.35%
Точность (полная):    99.80%

=== Веса (фиксированный шаг) ===
w1 = -8.1878
w2 = -8.1912
w3 = -8.1912
w4 = -8.1913
w5 = -8.1913
w6 = -8.1912
w7 = -8.1912
w8 = -2.5958
w9 = -8.1912
b = 4.5491
```

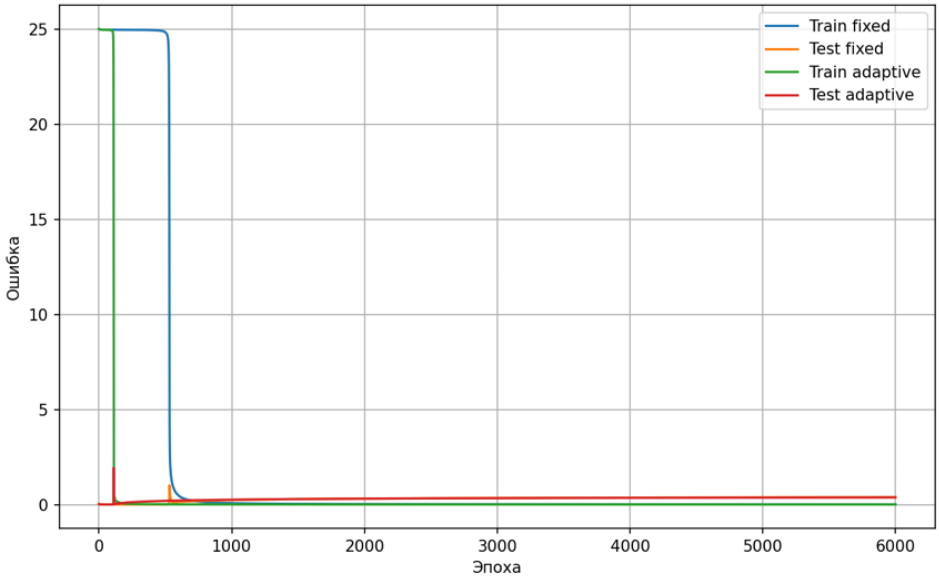
```
=== Веса (адаптивный шаг) ===
w1 = -10.2263
w2 = -10.2267
w3 = -10.2267
w4 = -10.2267
w5 = -10.2267
w6 = -10.2266
w7 = -10.2267
w8 = -3.7067
w9 = -10.2267
b = 5.5606

График сохранён: convergence.png

=== РЕЖИМ РАБОТЫ ===
Введите 9 чисел (0 или 1), либо q для выхода
Вход: 0 0 0 0 0 0 0 0 0
Вероятность: 0.9895
Класс: 1 | Истина: 1

Вход: 0 1 1 0 1 0 1 0 0
Вероятность: 0.0000
Класс: 0 | Истина: 0

Вход: 11
Ошибка количества значений
Вход: q
```



На графике видно, что при фиксированном шаге обучение идет медленнее, но ошибка постепенно уменьшается. При адаптивном шаге сеть обучается быстрее и быстрее сходится к хорошему результату. Ошибки на обучающей и тестовой выборках почти одинаковые, значит модель в целом нормально обобщает данные. При этом высокая точность объясняется тем, что данные сильно несбалансированы, так как почти все примеры относятся к одному классу.

Вывод: реализовала нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнила эффективность обучения при различных начальных условиях и параметрах шага.